

Sports Academy Management System (SAMS)

Full Technical Implementation Plan

Enterprise SaaS Multi-Tenant Architecture Blueprint

Version: 1.0 Date: May 2026 Target Audience: AI Engineering Agents, Technical Leads, Backend Engineers, Frontend Engineers, DevOps Engineers, QA Engineers, Product Owners

1. Executive Overview

This document defines the complete implementation strategy for building the Sports Academy Management System (SAMS) as a scalable, enterprise-grade, multi-tenant SaaS platform intended for deployment across sports academies throughout Egypt.

The purpose of this document is to provide an AI engineering agent or human development team with:

- Complete system architecture guidance
- Backend implementation structure
- Frontend implementation structure
- Infrastructure requirements
- Development sequencing
- Database design strategy
- API architecture
- Security standards
- Scalability planning
- DevOps workflows
- Testing requirements
- Deployment strategy
- Operational concerns
- SaaS multi-tenancy implementation
- Long-term maintainability guidelines

This implementation plan assumes:

- High concurrent user load
 - Multi-academy deployment
 - Production SaaS operation
 - Arabic + English support
 - Mobile-first workflows
 - Cloud-native infrastructure
 - Long-term extensibility
-

2. Product Strategy

2.1 Product Type

SAMS is NOT a simple internal management dashboard.

SAMS is:

- Multi-tenant SaaS platform
- Operational management system
- Scheduling engine
- Financial management system
- Attendance tracking platform
- Communication platform
- Analytics platform

The architecture MUST support:

- Hundreds of academies
 - Thousands of users
 - Millions of records
 - Real-time notifications
 - Future mobile applications
 - Future integrations
-

3. High-Level System Architecture

3.1 Architecture Style

Recommended architecture:

- Modular Monolith initially
- Service-oriented boundaries internally
- Future-ready for microservice extraction

Do NOT start with microservices.

Reason:

- Faster development
- Easier debugging
- Easier deployment
- Lower operational complexity
- Better consistency handling

The system should evolve toward service extraction only when scaling requires it.

3.2 Recommended Technology Stack

Backend

- Python 3.12+
- Django 5+
- Django REST Framework
- PostgreSQL 16+
- Redis
- Celery
- Django Channels (optional phase 2)
- JWT Authentication
- Docker
- NGINX
- Gunicorn

Frontend

- Vue 3
- Quasar Framework
- Pinia
- Axios
- Vue Router
- TypeScript
- Vue Query (recommended)

Infrastructure

- Docker Compose (development)
- Kubernetes (future scaling)
- AWS / Azure / GCP
- GitHub Actions
- Cloudflare
- S3-compatible object storage

Notifications

- SendGrid
- Twilio
- Firebase Push Notifications
- WhatsApp integration (future)

Monitoring

- Sentry
 - Prometheus
 - Grafana
 - ELK Stack (optional)
-

4. Multi-Tenant SaaS Architecture

4.1 Tenant Strategy

The platform MUST be designed as multi-tenant from day one.

Each academy is a tenant.

Create core entity:

```
Academy
- id
- name
- slug
- subscription_plan
- status
- branding_settings
- timezone
- language
- currency
- created_at
```

Every operational entity MUST include:

```
academy_id
```

Examples:

- User
- Player
- Coach
- Session
- Group
- Payment
- Attendance
- Reports
- Notifications

4.2 Tenant Isolation Rules

All queries MUST be academy-scoped.

Never expose data across academies.

Mandatory:

- Tenant middleware
- Tenant-aware managers/queriesets
- Tenant-aware permissions
- Tenant-aware caching
- Tenant-aware analytics

Recommended implementation:

```
class AcademyScopedModel(models.Model):  
    academy = models.ForeignKey(Academy)
```

All business models inherit from this.

5. Backend Architecture

5.1 Django Apps Structure

Recommended app structure:

```
apps/  
  academies/  
  accounts/  
  permissions/  
  players/  
  coaches/  
  groups/  
  sessions/  
  attendance/  
  reports/  
  ratings/  
  cancellations/  
  payments/  
  notifications/  
  analytics/  
  audit/  
  storage/  
  communication/  
  common/
```

Avoid:

```
core/  
main/
```

```
system/  
utils/
```

Domain-driven structure is required.

5.2 Accounts & Authentication

Custom User Model

Required.

Fields:

```
class User(AbstractBaseUser):  
    academy  
    role  
    email  
    phone  
    first_name  
    last_name  
    is_active  
    mfa_enabled
```

5.3 RBAC Design

Role hierarchy:

1. Customer
2. Coach
3. Operations
4. Admin
5. Super Admin (internal SaaS management)

Use:

- Django permissions
- Role enums
- Permission decorators
- DRF custom permissions

Never rely on frontend-only permissions.

6. Database Architecture

6.1 Database Choice

Mandatory:

- PostgreSQL

Avoid MySQL.

Reasons:

- Better concurrency
 - Better indexing
 - Better JSON support
 - Better reporting
 - Better future analytics
-

6.2 Database Standards

Mandatory Standards

- UUID primary keys
- created_at
- updated_at
- soft deletes
- indexed foreign keys
- transactional writes
- database constraints

Example:

```
id = models.UUIDField(primary_key=True)
```

6.3 Critical Indexing

Indexes required on:

- academy_id
- created_at
- user_id
- session_id
- player_id
- coach_id
- payment_status

- session_datetime

Composite indexes:

```
(academy_id, created_at)
(academy_id, status)
(academy_id, session_datetime)
```

7. Scheduling Engine Architecture

7.1 Scheduling Complexity

Scheduling is the MOST COMPLEX subsystem.

Must support:

- Recurring sessions
- Single overrides
- Multi-coach sessions
- Venue conflicts
- Capacity limits
- Player conflicts
- Coach conflicts
- Cancellation workflows
- Session cloning
- Timezone support

This subsystem MUST be isolated.

7.2 Session Model Design

Core entities:

```
SessionSeries
SessionOccurrence
SessionEnrollment
Venue
ScheduleConflict
```

Avoid storing recurrence logic directly on simple session rows.

7.3 Recurring Sessions Strategy

Use recurrence rules.

Recommended:

- RFC 5545 recurrence patterns
- python-dateutil

Never duplicate recurring sessions manually.

Generate occurrences dynamically.

8. Payments Architecture

8.1 Payment Requirements

The Egyptian market requires:

- Cash support
 - Bank transfer support
 - Online payment support
 - Manual reconciliation
 - Installment support (future)
 - Partial payment support
-

8.2 Payment Providers

Recommended:

- Paymob
- PayTabs Egypt
- Fawry

Stripe should not be primary for Egypt.

8.3 Payment Models

Required entities:

```
Invoice
Payment
PaymentTransaction
Refund
```

Discount
Coupon
LateFee

8.4 Financial Safety

Financial records MUST NEVER be hard deleted.

Use:

- Immutable transaction logs
 - Audit trails
 - Double validation
 - Admin approvals
-

9. Notification Architecture

9.1 Notifications MUST Be Asynchronous

Never send notifications synchronously.

All notification tasks MUST use:

- Celery
 - Redis queues
-

9.2 Notification Channels

Required:

- Email
- SMS
- Push notifications
- In-app notifications

Future:

- WhatsApp
-

9.3 Notification Pipeline

Flow:

Trigger Event

- Queue Job
- Notification Service
- Channel Adapter
- Delivery Tracking

10. File Storage Architecture

10.1 Storage Requirements

Store:

- Profile photos
- Reports
- Exports
- Attachments
- Generated PDFs

Use:

- AWS S3
- Cloudflare R2
- MinIO (development)

Never store uploaded files on local disk in production.

11. Frontend Architecture

11.1 Frontend Strategy

Use:

- Vue 3
- Quasar
- TypeScript

Architecture goals:

- Mobile-first
 - Responsive
 - RTL support
 - Component-driven
 - API-first
-

11.2 Frontend Folder Structure

```
src/  
  api/  
  boot/  
  components/  
  composables/  
  layouts/  
  pages/  
  router/  
  stores/  
  types/  
  utils/
```

11.3 State Management

Use Pinia.

Separate stores:

```
authStore  
sessionStore  
paymentStore  
notificationStore  
analyticsStore
```

12. Mobile Strategy

12.1 Mobile Applications

Do NOT build native apps initially.

Use:

- Quasar Capacitor

Benefits:

- Shared codebase
 - Faster development
 - Lower cost
 - Easier maintenance
-

13. API Design Standards

13.1 API Style

Use REST API.

Future GraphQL support optional.

13.2 API Standards

Requirements:

- Versioned APIs
- JWT auth
- Pagination
- Filtering
- Search
- Sorting
- Rate limiting
- OpenAPI documentation

Example:

```
/api/v1/players/  
/api/v1/sessions/
```

13.3 API Response Format

Standardized response:

```
{  
  "success": true,  
  "data": {},  
  "message": ""  
}
```

14. Security Architecture

14.1 Security Requirements

Mandatory:

- HTTPS only
 - TLS 1.3
 - bcrypt password hashing
 - MFA support
 - JWT rotation
 - Role-based access
 - Tenant isolation
 - Rate limiting
 - IP logging
 - Audit logging
-

14.2 Sensitive Data Handling

Encrypt:

- Medical notes
- Sensitive payment metadata
- Personal identifiers

Use field-level encryption where necessary.

15. Audit Logging System

15.1 Audit Logging Requirements

Every critical action MUST be logged.

Track:

- Actor
 - Action
 - Entity
 - Old value
 - New value
 - IP address
 - Timestamp
-

16. Background Processing

16.1 Celery Requirements

Mandatory queues:

```
notifications
emails
sms
exports
reports
analytics
payments
cleanup
```

16.2 Scheduled Jobs

Use Celery Beat.

Scheduled tasks:

- Reminder notifications
- Late fee processing
- Cleanup jobs
- Analytics refresh
- Report generation
- Backup verification

17. Analytics Architecture

17.1 Analytics Separation

Never run heavy analytics directly on operational tables indefinitely.

Recommended:

- Materialized views
- Aggregated analytics tables
- Scheduled analytics jobs

17.2 Analytics KPIs

Track:

- Retention
 - Attendance rates
 - Revenue
 - Session utilization
 - Churn
 - Coach performance
 - Enrollment trends
-

18. Reporting System

18.1 Export Support

Support:

- PDF
- Excel
- CSV

Large reports MUST be asynchronous.

18.2 Reporting Strategy

Never generate heavy reports synchronously.

Flow:

```
Generate Request
→ Queue Job
→ Generate File
→ Upload to Storage
→ Notify User
```

19. DevOps Architecture

19.1 Development Environments

Required environments:

- Local
 - Development
 - Staging
 - Production
-

19.2 Docker Setup

Services:

```
nginx
backend
frontend
postgres
redis
celery
celery-beat
minio
```

19.3 CI/CD

Use GitHub Actions.

Pipeline:

```
Lint
→ Tests
→ Build
→ Security Checks
→ Deploy
```

20. Infrastructure Architecture

20.1 Cloud Requirements

Recommended:

- AWS
- Azure
- GCP

Initial deployment:

- Single-region
 - Managed PostgreSQL
 - Redis
 - Object storage
-

20.2 Scalability Strategy

The architecture MUST support:

- Horizontal scaling
 - Stateless backend
 - Queue scaling
 - Read replicas (future)
 - CDN support
-

21. Monitoring & Observability

21.1 Monitoring Stack

Required:

- Sentry
 - Prometheus
 - Grafana
 - Structured logging
-

21.2 Metrics

Track:

- API latency
- Queue latency

- DB performance
 - Error rates
 - Notification failures
 - Login failures
 - Payment failures
-

22. Internationalization (i18n)

22.1 Language Requirements

Mandatory:

- Arabic
- English
- RTL/LTR support

Use:

- Django i18n
 - Vue i18n
-

23. Accessibility

23.1 WCAG Compliance

Target:

- WCAG 2.1 AA

Requirements:

- Keyboard navigation
 - Color contrast
 - Screen reader support
 - Semantic HTML
-

24. Testing Strategy

24.1 Backend Testing

Required:

- Unit tests
- Integration tests

- Permission tests
- Tenant isolation tests
- Payment tests
- API tests

Coverage target:

- Minimum 80%
-

24.2 Frontend Testing

Required:

- Component tests
- E2E tests
- Accessibility tests

Recommended:

- Vitest
 - Playwright
-

25. QA Strategy

25.1 Manual QA

Critical flows:

- Login
 - Scheduling
 - Payments
 - Attendance
 - Notifications
 - Cancellations
 - Reporting
-

26. Data Migration Strategy

26.1 Academy Onboarding

Many academies will migrate from:

- Excel
- Google Sheets
- Paper systems

Build:

- CSV import system
 - Validation engine
 - Error reports
 - Bulk onboarding workflows
-

27. SaaS Billing Architecture

27.1 Subscription System

The platform itself requires billing.

Entities:

```
SubscriptionPlan
Subscription
Invoice
TenantUsage
```

27.2 Plan Restrictions

Possible limits:

- Player count
 - Coach count
 - Storage
 - SMS usage
 - Feature access
-

28. AI Agent Development Workflow

28.1 Recommended AI Agent Execution Order

The AI development agent SHOULD build the project in the following order:

Phase 1

Infrastructure Foundation

1. Repository setup
2. Docker setup
3. Django setup

4. PostgreSQL setup
 5. Redis setup
 6. Celery setup
 7. Authentication setup
 8. Multi-tenancy setup
 9. RBAC setup
 10. CI/CD setup
-

Phase 2

Core Data Layer

1. Academy models
 2. User models
 3. Player models
 4. Coach models
 5. Group models
 6. Session models
 7. Enrollment models
 8. Attendance models
 9. Audit models
-

Phase 3

Operations Portal

1. CRUD dashboards
 2. Scheduling system
 3. Conflict detection
 4. Group management
 5. Attendance workflows
 6. Session management
-

Phase 4

Customer Portal

1. Authentication
 2. Timetable
 3. Profile management
 4. Session history
 5. Notifications
 6. Cancellation requests
-

Phase 5

Coach Portal

1. Coach timetable
 2. Attendance marking
 3. Session reports
 4. Player ratings
 5. Notifications
-

Phase 6

Payments

1. Invoices
 2. Manual payments
 3. Online payments
 4. Reconciliation
 5. Refunds
 6. Reporting
-

Phase 7

Notifications

1. Email engine
 2. SMS engine
 3. Push notifications
 4. In-app notifications
 5. Reminder jobs
-

Phase 8

Analytics & Reporting

1. Dashboards
 2. KPIs
 3. Financial reporting
 4. Export systems
 5. Scheduled reports
-

Phase 9

Advanced Features

1. Messaging

2. Achievement system
 3. Forecasting
 4. Mobile apps
 5. WhatsApp integration
-

29. Recommended Git Strategy

29.1 Branching

Use:

```
main
develop
feature/*
hotfix/*
```

29.2 Commit Standards

Use conventional commits:

```
feat:
fix:
refactor:
test:
chore:
```

30. Performance Optimization Strategy

30.1 Backend Optimization

Required:

- select_related
 - prefetch_related
 - Redis caching
 - query optimization
 - pagination
 - async tasks
-

30.2 Frontend Optimization

Required:

- lazy loading
 - route splitting
 - image optimization
 - virtual scrolling
 - request caching
-

31. Disaster Recovery

31.1 Backup Strategy

Mandatory:

- Daily DB backups
 - Object storage backups
 - Backup validation
 - Recovery testing
-

32. Production Deployment Strategy

32.1 Initial Deployment

Recommended architecture:

```
Cloudflare
→ NGINX
→ Django API
→ PostgreSQL
→ Redis
→ Celery Workers
→ Object Storage
```

33. Long-Term Evolution Plan

33.1 Future System Evolution

Potential future services:

- Dedicated analytics service
 - Messaging microservice
 - AI recommendation engine
 - Video coaching platform
 - QR attendance service
 - AI retention prediction
-

34. Critical Engineering Rules

34.1 Mandatory Rules

1. NEVER hardcode academy-specific logic
 2. NEVER trust frontend permissions
 3. NEVER run heavy reports synchronously
 4. NEVER send notifications synchronously
 5. NEVER expose cross-tenant data
 6. NEVER tightly couple scheduling logic
 7. NEVER store files locally in production
 8. NEVER skip audit logging
 9. NEVER allow hard deletion of financial data
 10. NEVER skip automated tests
-

35. Recommended Initial Team Structure

35.1 Core Team

Recommended:

- 2 Backend Engineers
 - 1 Frontend Engineer
 - 1 DevOps Engineer
 - 1 QA Engineer
 - 1 Product Owner
-

36. Estimated Technical Complexity

36.1 Complexity Assessment

Subsystem complexity:

System	Complexity
Authentication	Medium
RBAC	High
Scheduling	Very High
Payments	High
Notifications	Medium
Reporting	High
Analytics	High
Multi-tenancy	Very High
Mobile Support	Medium

37. Recommended Delivery Timeline

37.1 Timeline

Months 1-2

- Infrastructure
- Authentication
- Multi-tenancy
- Core models

Months 3-4

- Scheduling
- Operations portal
- Attendance

Months 5-6

- Customer portal
- Coach portal
- Notifications

Months 7-8

- Payments
- Reporting
- Analytics

Months 9-10

- Optimization
- QA
- Security hardening
- Mobile packaging

38. Final Engineering Guidance

This system MUST prioritize:

1. Operational reliability
2. Data integrity
3. Multi-tenant isolation
4. Scalability
5. Maintainability
6. Mobile usability
7. Performance
8. Simplicity of operations

The project should be treated as a long-term SaaS platform, not as a temporary custom dashboard.

All architecture decisions should optimize for:

- Maintainability
- Extensibility
- Reliability
- Operational simplicity
- Scalability

The most critical subsystem is the scheduling engine.

The most critical business feature is operational efficiency.

The most critical engineering concern is tenant isolation.

The most critical scaling concern is asynchronous processing.

39. AI AGENT MASTER PROMPT SYSTEM

The following section contains production-grade prompts intended to be used directly with AI engineering agents.

These prompts are designed to:

- enforce architectural consistency
- reduce hallucinated implementations
- maintain enterprise standards
- ensure scalability
- preserve multi-tenant safety
- standardize development workflow

Each prompt should be executed sequentially.

The AI agent **MUST** complete and validate one phase before moving to the next.

PROMPT 1 — MASTER SYSTEM ARCHITECT PROMPT

Purpose

This prompt establishes global engineering rules and architectural constraints for the entire project.

Prompt

You are a senior enterprise software architect and lead engineer responsible for building a production-grade multi-tenant SaaS platform called SAMS (Sports Academy Management System).

The system will serve sports academies across Egypt and must support:

- multi-tenancy
- high concurrency
- financial workflows
- scheduling
- notifications
- analytics
- mobile-first operation
- Arabic and English support
- enterprise security standards

You must follow these architectural rules strictly:

1. Backend Stack

- Python 3.12+
- Django 5+
- Django REST Framework
- PostgreSQL
- Redis
- Celery
- JWT authentication
- Dockerized infrastructure

2. Frontend Stack

- Vue 3
- Quasar Framework
- TypeScript
- Pinia
- Axios

3. Architecture Rules

- Multi-tenant SaaS from day one
- Every operational entity must belong to an academy tenant
- Never expose cross-tenant data
- Use UUID primary keys
- Use soft deletes where applicable
- Use asynchronous processing for notifications and heavy tasks
- Never tightly couple business domains
- Maintain modular domain-driven app architecture
- Build mobile-first responsive UI
- Support Arabic RTL and English LTR

4. Security Rules

- RBAC enforced on backend
- JWT token rotation
- HTTPS-only architecture
- Audit logging required
- MFA-ready architecture
- Financial records must never be hard deleted

5. Scalability Rules

- Horizontally scalable backend
- Queue-based architecture for background jobs
- Redis caching support
- Optimized PostgreSQL indexing
- Async notifications and report generation

6. Code Quality Rules

- Clean architecture
- Type-safe frontend
- Service-layer business logic
- Repository/query abstraction when necessary
- Comprehensive tests

- No duplicated logic
- Proper serializer validation
- Reusable frontend composables/components

7. Development Rules

- Generate production-ready code only
- Never generate placeholder architecture
- Never skip validation
- Never skip permissions
- Never trust frontend permissions
- Never use synchronous heavy processing

Always explain:

- architectural decisions
- model relationships
- permission strategy
- scaling considerations
- optimization strategy

All future prompts inherit these rules.

PROMPT 2 — BACKEND FOUNDATION SETUP

Purpose

Initialize the backend architecture and infrastructure.

Prompt

Build the complete backend foundation for SAMS using Django.

Requirements:

1. Create production-ready Dockerized backend architecture.

2. Setup services:

- Django
- PostgreSQL
- Redis
- Celery
- Celery Beat
- NGINX

3. Configure:

- environment variables
- settings separation

- logging
- CORS
- JWT auth
- Redis caching
- Celery queues
- static/media handling

4. Create modular app structure:

```
apps/  
  academies/  
  accounts/  
  permissions/  
  players/  
  coaches/  
  groups/  
  sessions/  
  attendance/  
  reports/  
  ratings/  
  cancellations/  
  payments/  
  notifications/  
  analytics/  
  audit/  
  communication/  
  common/
```

5. Implement:

- custom user model
- academy tenant model
- RBAC structure
- audit middleware
- base abstract models
- UUID primary keys
- timestamp mixins
- soft delete mixins

6. Configure:

- DRF
- JWT authentication
- API versioning
- pagination
- filtering
- standardized API responses

7. Setup:

- pre-commit hooks
- linting
- formatting
- pytest

- CI-ready structure

Generate:

- complete folder structure
- Docker files
- environment templates
- Django settings
- requirements
- startup commands
- architecture explanations

PROMPT 3 — MULTI-TENANCY IMPLEMENTATION

Purpose

Implement enterprise-grade tenant isolation.

Prompt

Implement multi-tenant architecture for SAMS.

Requirements:

1. Create academy tenant system.
2. Every operational model must include academy ownership.
3. Build:
 - Academy model
 - tenant middleware
 - tenant-aware managers
 - tenant-aware querysets
 - tenant-aware permissions
4. Ensure:
 - no cross-tenant data leaks
 - automatic academy filtering
 - tenant-safe APIs
 - tenant-safe caching
 - tenant-safe analytics
5. Implement:
 - abstract academy-scoped model
 - automatic tenant injection
 - tenant validation

6. Add tests for:
- tenant isolation
 - query protection
 - permission boundaries
 - malicious access attempts

7. Explain:
- scaling implications
 - query optimization
 - security implications
 - future shardability

PROMPT 4 — AUTHENTICATION & RBAC SYSTEM

Purpose

Build enterprise authentication and permissions.

Prompt

Build the complete authentication and RBAC system for SAMS.

Requirements:

Roles:

- Customer
- Coach
- Operations
- Admin
- Super Admin

Implement:

1. Authentication

- JWT login
- refresh tokens
- logout
- password reset
- email verification
- MFA-ready architecture

2. RBAC

- backend-enforced permissions
- role hierarchy
- permission decorators
- DRF permissions

- object-level access

3. Security

- password policies
- login throttling
- suspicious login logging
- audit logging
- session expiration

4. Build APIs:

- register
- login
- logout
- refresh token
- forgot password
- profile
- role management

5. Create:

- serializers
- services
- permissions
- tests
- documentation

6. Explain:

- security design
- token lifecycle
- permission enforcement strategy

PROMPT 5 — SCHEDULING ENGINE IMPLEMENTATION

Purpose

Build the most critical subsystem.

Prompt

Build the enterprise scheduling engine for SAMS.

This is the most critical subsystem.

Requirements:

1. Build models:
 - SessionSeries
 - SessionOccurrence
 - Venue
 - Enrollment
 - SessionCoach
 - ScheduleConflict
2. Support:
 - recurring sessions
 - custom recurrence rules
 - single occurrence overrides
 - multi-coach sessions
 - venue management
 - player enrollments
 - conflict detection
 - cancellation workflows
 - capacity enforcement
3. Implement:
 - RFC5545 recurrence handling
 - conflict detection engine
 - schedule validation
 - recurring occurrence generation
 - timezone-safe scheduling
4. Build APIs:
 - create recurring session
 - update occurrence
 - cancel occurrence
 - enroll players
 - assign coaches
 - fetch timetables
5. Optimize for:
 - large datasets
 - high concurrency
 - minimal query count
 - indexed datetime lookups
6. Add:
 - transaction safety
 - optimistic locking where needed
 - concurrency protection
7. Write:
 - extensive tests
 - performance-safe queries
 - scheduling services
 - architecture documentation

Explain all scheduling decisions carefully.

PROMPT 6 — OPERATIONS PORTAL BACKEND

Purpose

Build operations management functionality.

Prompt

Build the Operations Portal backend for SAMS.

Features:

1. Player management
 - CRUD
 - bulk imports
 - filtering
 - searching
 - enrollment management
2. Coach management
 - CRUD
 - availability
 - assignments
 - workload tracking
3. Group management
 - create groups
 - assign players
 - assign sessions
 - roster analytics
4. Session management
 - scheduling integration
 - approvals
 - cancellation review
5. Attendance management
 - real-time attendance
 - overrides
 - reporting
6. Communication tools
 - announcements

- targeted messaging
- delivery logs

7. Build:

- services
- serializers
- APIs
- permissions
- tests
- filtering
- pagination

8. Optimize:

- query efficiency
- bulk operations
- transactional integrity

PROMPT 7 — CUSTOMER PORTAL IMPLEMENTATION

Purpose

Build customer-facing functionality.

Prompt

Build the Customer Portal backend and frontend for SAMS.

Requirements:

1. Timetable views

- weekly calendar
- monthly calendar
- session details
- filtering
- color-coded statuses

2. Player profile

- ratings
- progress charts
- achievements
- attendance history

3. Session cancellation requests

- policy validation

- deadline enforcement
- automated approvals
- operations review workflow

4. Notifications center

- read/unread state
- reminders
- cancellations
- payment alerts

5. Parent support

- multiple child profiles
- linked accounts
- centralized payments

6. Build:

- responsive Quasar UI
- Pinia stores
- API integrations
- composables
- tests

7. Ensure:

- mobile-first UX
- RTL support
- accessibility
- optimized API usage

PROMPT 8 — COACH PORTAL IMPLEMENTATION

Purpose

Build coach workflows.

Prompt

Build the Coach Portal backend and frontend for SAMS.

Requirements:

1. Coach timetable

- daily/weekly views
- player lists
- session details

2. Attendance workflows

- mark attendance
- late status
- absences
- attendance summaries

- 3. Session reports
 - report submission
 - report deadlines
 - historical archive
 - aggregate statistics

- 4. Player ratings
 - skill ratings
 - performance notes
 - feedback visibility

- 5. Cancellation requests
 - coach cancellation workflow
 - mandatory reason
 - operations approval

- 6. Notifications
 - reminders
 - cancellation alerts
 - player messages

- 7. Build:
 - backend APIs
 - frontend pages
 - optimized data fetching
 - reusable UI components
 - tests

PROMPT 9 — PAYMENTS & FINANCIAL SYSTEM

Purpose

Build the complete offline-first financial management infrastructure for SAMS.

IMPORTANT:

This platform DOES NOT support online payment processing.

Do NOT integrate:

- Stripe
- Paymob
- PayTabs

- Fawry
- payment gateways
- card processing
- online checkout systems
- payment webhooks

The system is strictly designed for:

- cash collection tracking
- bank transfer tracking
- offline payment reconciliation
- financial reporting
- payment obligation management

Prompt

Build the complete offline-first payment and financial management system for SAMS.

IMPORTANT:

This platform does NOT support online payments.

Do NOT implement:

- payment gateways
- card processing
- online checkout
- webhooks
- transaction processors
- stored card data
- PCI payment flows

Requirements:

1. Payment entities:

- Invoice
- Payment
- Refund
- Discount
- Coupon
- LateFee
- PaymentInstallment
- FinancialAdjustment

2. Support:

- cash payments
- bank transfer recording
- partial payments
- installment plans

- manual reconciliation
- refunds
- overdue tracking
- payment obligations
- payment receipts

3. Build:

- financial dashboards
- receivable tracking
- overdue reporting
- payment history
- payment summaries
- export support

4. Administrative Features:

- payment approval workflows
- adjustment approvals
- refund approvals
- financial notes
- receipt generation
- audit trails

5. Security:

- immutable financial logs
- audit logging
- admin approval workflows
- no hard deletion of financial records

6. Optimize:

- transactional integrity
- concurrency safety
- financial consistency
- reporting performance

7. Add:

- tests
- reconciliation workflows
- reporting queries
- async exports

8. Generate:

- Django models
- serializers
- services
- APIs
- reporting architecture
- permission system
- audit system
- financial workflow documentation

9. Explain:

- financial consistency strategy
- audit strategy
- reconciliation logic
- installment logic
- reporting optimization strategy

PROMPT 10 — NOTIFICATION SYSTEM

Purpose

Build asynchronous communications.

Prompt

Build the asynchronous notification infrastructure for SAMS.

Requirements:

1. Notification channels:

- Email
- SMS
- Push notifications
- In-app notifications

2. Use:

- Celery
- Redis queues

3. Build:

- notification service layer
- channel adapters
- delivery tracking
- retry handling
- failure logging

4. Support:

- reminders
- cancellations
- payment alerts
- announcements
- system notifications

5. Implement:

- templates
- localization
- scheduling

- throttling

6. Add:

- async jobs
- monitoring
- metrics
- tests

PROMPT 11 — ANALYTICS & REPORTING

Purpose

Build operational intelligence systems.

Prompt

Build the analytics and reporting system for SAMS.

Requirements:

1. Analytics dashboards:

- revenue
- attendance
- retention
- utilization
- coach performance
- enrollment trends

2. Reporting:

- PDF exports
- Excel exports
- CSV exports
- scheduled reports

3. Build:

- materialized views
- analytics aggregation tables
- async report generation
- scheduled analytics refresh jobs

4. Optimize:

- large datasets
- low-latency dashboards
- minimal DB locking

5. Add:

- financial reporting
- operational reporting
- scheduled email delivery
- tests

PROMPT 12 — FRONTEND SYSTEM ARCHITECTURE

Purpose

Build scalable frontend architecture.

Prompt

Build the frontend architecture for SAMS using Vue 3 and Quasar.

Requirements:

1. Architecture:

- modular structure
- TypeScript
- reusable components
- composables
- Pinia stores
- route-based code splitting

2. Build:

- layouts
- navigation system
- auth flows
- dashboard framework
- responsive pages

3. Implement:

- RTL/LTR switching
- i18n
- accessibility
- mobile-first UX

4. Optimize:

- API usage
- request caching
- lazy loading
- rendering performance

- 5. Create:
 - design system
 - reusable forms
 - reusable tables
 - reusable dialogs
 - calendar components

- 6. Add:
 - testing
 - type safety
 - documentation

PROMPT 13 — DEVOPS & DEPLOYMENT

Purpose

Build production infrastructure.

Prompt

Build the DevOps and deployment infrastructure for SAMS.

Requirements:

- 1. Dockerize:
 - backend
 - frontend
 - PostgreSQL
 - Redis
 - Celery
 - NGINX
- 2. Configure:
 - GitHub Actions
 - CI/CD pipelines
 - automated testing
 - linting
 - deployments
- 3. Infrastructure:
 - production environment
 - staging environment
 - secrets management
 - backup automation
- 4. Monitoring:

- Sentry
- Prometheus
- Grafana
- structured logs

5. Security:

- HTTPS
- firewall strategy
- container hardening
- environment isolation

6. Add:

- deployment scripts
- rollback strategy
- backup recovery procedures
- scaling guidance

PROMPT 14 — TESTING & QA SYSTEM

Purpose

Build enterprise testing infrastructure.

Prompt

Build the testing and QA infrastructure for SAMS.

Requirements:

1. Backend testing:

- unit tests
- integration tests
- permission tests
- tenant isolation tests
- API tests
- payment tests

2. Frontend testing:

- component tests
- E2E tests
- accessibility tests
- mobile responsiveness tests

3. Build:

- test factories
- fixtures

- mocking utilities
 - CI integration
4. Coverage:
- minimum 80% backend coverage
 - critical workflow E2E coverage
5. Add:
- performance testing
 - concurrency testing
 - load testing
 - security testing

PROMPT 15 — FINAL PRODUCTION HARDENING

Purpose

Prepare the platform for production SaaS deployment.

Prompt

Perform final production hardening for SAMS.

Requirements:

1. Security review
2. Performance optimization
3. Query optimization
4. Caching review
5. Audit logging validation
6. Tenant isolation validation
7. Backup testing
8. Load testing
9. Accessibility review
10. Mobile responsiveness review
11. Error handling review
12. Monitoring validation
13. CI/CD validation
14. Documentation completion
15. Disaster recovery validation

Generate:

- production readiness checklist
- infrastructure checklist
- security checklist
- deployment runbook

- maintenance procedures
 - incident response procedures
-

40. AI AGENT EXECUTION RULES

Mandatory Rules

1. Complete one prompt fully before moving to the next.
 2. Never skip tests.
 3. Never skip permissions.
 4. Never generate pseudo-code unless explicitly requested.
 5. Generate production-ready code only.
 6. Always explain architectural decisions.
 7. Always optimize for scalability.
 8. Always preserve tenant isolation.
 9. Always prefer maintainability over cleverness.
 10. Always validate security implications.
-

END OF DOCUMENT